

BCA182

Embedded Systems Programming Laboratory

LABORATORY ACTIVITY 1

Real-Time Multisensor Room Monitoring System

Laboratory Report

Prepared by:

Charles Daniel Estrada

Platform: STM32F103C8T6 Blue Pill - PlatformIO - STM32Cube HAL - FreeRTOS

Date: September 2026

Verification note

This report distinguishes implemented functionality from behavior that was actually observed. Wokwi startup was observed, but DHT22 communication, OLED output, encoder interaction, buzzer activation, and serial output beyond startup were not treated as reliably runtime-verified.

1. Problem and Requirements

The laboratory activity develops a real-time multisensor room monitoring system using an STM32F103C8T6 Blue Pill and FreeRTOS. The system acquires temperature, humidity, ambient-light, and motion information, presents a selected measurement on an SSD1306 OLED, accepts rotary-encoder navigation, controls a buzzer for temperature alarms, and maintains an ACTIVE/INACTIVE room state.

The implementation follows the laboratory constraints: C/C++, STM32 HAL, native FreeRTOS APIs, PlatformIO, and no Arduino framework. The final project is organized into incremental milestones and tracked using Git.

Functional Requirements

Requirement	Implementation
FR-01 / FR-02	DHT22 temperature and humidity acquisition
FR-03	LDR relative ambient-light level, 0-100
FR-04	PIR motion detection
FR-05 / FR-06	OLED display with rotary-encoder measurement selection
FR-07	Temperature alarm below 18 C or above 30 C
FR-08 / FR-09 / FR-10	ACTIVE/INACTIVE state machine driven by motion and inactivity timeout

2. System Architecture and Design

The system is divided into sensor acquisition, motion/state management, user input, display, and alarm functions. Sensor readings are packaged in a SensorData structure and transferred through a FreeRTOS queue. The design uses task notifications for motion-to-state communication and mutexes for shared resources such as the display-selection state.

Pin Mapping

Device / Function	Pin	Purpose
LDR	PA0 / ADC1 CH0	Relative ambient-light level
DHT22	PA1	Temperature and humidity
PIR	PB0	Motion detection
Buzzer	PB1	Temperature alarm
OLED	PB6 / PB7	I2C1 SCL / SDA
Rotary encoder	PB10 / PB11	CLK / DT navigation
On-board LED	PC13	Basic task/board indication
USART1	PA9 / PA10	TX / RX

3. FreeRTOS Architecture

The application uses multiple FreeRTOS tasks with explicit priorities and blocking delays. SensorTask and DisplayTask use vTaskDelayUntil() to maintain periodic execution without drift. MotionTask polls the PIR every 100 ms, while StateTask waits for a task notification and applies the inactivity rule.

Task Table

Task	Priority	Stack	Period / Timeout	Main responsibility
SensorTask	2	256 words	2000 ms	DHT22/LDR acquisition and sensor queue (vTaskDelayUntil)
DisplayTask	2	352 words	200 ms / 500 ms	OLED rendering from latest sensor data (vTaskDelayUntil)
InputTask	3	256 words	1 ms poll	Rotary encoder navigation
MotionTask	3	256 words	100 ms	PIR polling and motion notification
AlarmTask	2	256 words	500 ms	Temperature threshold and buzzer
StateTask	2	256 words	15 s timeout	ACTIVE/INACTIVE state machine
TaskA	2	256 words	1000 ms	Toggles onboard LED (PC13), prints diagnostic via UART
TaskB	1	256 words	1500 ms	Prints diagnostic message via UART

Inter-Task Communication and Synchronization

The application employs several FreeRTOS IPC and synchronization mechanisms to ensure correct data sharing and task coordination.

Sensor Queue

A FreeRTOS queue of depth 4 carrying SensorData structs. SensorTask is the sole producer via xQueueSend, non-blocking with zero tick count; samples are dropped when full. DisplayTask and AlarmTask are consumers and use xQueuePeek for non-destructive reads so the latest sample is available without consuming the item.

```
struct SensorData {  
    float temperature; // DHT22 reading (C)  
    float humidity; // DHT22 reading (%)  
    int lightLevel; // LDR ADC scaled 0-100  
    bool motionDetected; // PIR state snapshot  
    bool dhtValid; // true only when DHT22_Read succeeded  
};
```

Mutexes

Mutex	Created in	Protects
serialMutex	main.cpp	HAL_UART_Transmit - prevents multi-task serial output from interleaving via Serial_Print()
displayModeMutex	input.cpp	selectedDisplayMode - InputTask writes, DisplayTask reads via Input_GetDisplayMode()

Task Notification

MotionTask -> StateTask via xTaskNotifyGive / ulTaskNotifyTake. MotionTask calls SystemState_NotifyMotionDetected() each time the PIR pin reads HIGH. StateTask blocks with ulTaskNotifyTake(pdTRUE, 15000 ms): a notification sets ACTIVE, while a 15-second timeout sets INACTIVE.

3. FreeRTOS Architecture (continued)

Critical Sections

`taskENTER_CRITICAL()` / `taskEXIT_CRITICAL()` protect the shared `activityState` and `motionDetected` variables in `SystemState_GetActivityState()`, `SystemState_IsMotionDetected()`, and the `SetState()` helper. These short critical sections disable interrupts to ensure atomic reads and writes on the Cortex-M3.

`vTaskDelayUntil()` Usage

`vTaskDelayUntil()` is used by `SensorTask` (2000 ms period) and `DisplayTask` (200 ms in ACTIVE mode, 500 ms in INACTIVE mode) to maintain drift-free periodic execution. This API is preferred over `vTaskDelay()` for periodic tasks because it compensates for the execution time of the task body.

4. Implementation

4.1 Milestones 1-2: Sensors and Activity State

Milestone 1 implemented the DHT22 and LDR subsystem. The DHT22 driver performs the required GPIO timing and validates received data using the checksum. The LDR uses ADC1 channel 0 and converts the raw reading to a relative 0-100 light level rather than calibrated lux. `SensorData` contains temperature, humidity, light level, motion status, and a `dhtValid` flag, and `SensorTask` places valid readings into a four-element queue.

Milestone 2 added the PIR and activity state machine. `MotionTask` runs at priority 3 with a 100 ms polling interval. `StateTask` starts ACTIVE, changes to INACTIVE after 15 seconds without motion, and returns to ACTIVE when `MotionTask` sends a task notification.

4.2 Milestones 3-5: Input, Display, and Alarm

Milestone 3 added rotary-encoder navigation using PB10 and PB11. `InputTask` polls the encoder at 1 ms intervals with debouncing and protects the selected `DisplayMode` with a mutex. Navigation wraps around the four display modes and is ignored while the room is INACTIVE.

Milestone 4 added a bare-metal SSD1306 I2C driver using STM32 HAL, avoiding the Arduino framework. `DisplayTask` runs periodically (200 ms ACTIVE, 500 ms INACTIVE), reads the latest sensor data without consuming the queue item, obtains the selected display mode and activity state, and renders the corresponding value.

Milestone 5 added `AlarmTask` and a buzzer on PB1. The alarm activates when temperature is below 18 C or above 30 C and is silenced while the system is INACTIVE. `AlarmTask` checks the `dhtValid` flag to prevent false alarms when DHT22 communication fails.

4.3 Milestone 6: Unit Testing

The project includes a native Unity-based test environment for logic that can be tested without the STM32 hardware. Three test suites cover temperature evaluation, display-mode navigation, and system-state evaluation. The suite contains 13 meaningful test cases in total. A portable MinGW-w64 host compiler and small HAL/FreeRTOS mocks were used only for the native test build; the production STM32 architecture was not replaced by a native implementation.

Test suite	Cases	Coverage
Temperature evaluation	5	Normal, lower boundary, upper boundary, below range, above range
Display mode navigation	4	All four measurement modes and wraparound behavior
System state evaluation	4	ACTIVE/INACTIVE transitions and motion/inactivity conditions
Total	13	13 meaningful unit tests

5. Verification and Testing

Verification was performed through firmware compilation, native unit testing, static analysis, configuration inspection, and available Wokwi simulation behavior. The project also maintains incremental Git commits for the completed milestones.

Verification item	Result
STM32 PlatformIO build	Successful; no compiler errors or warnings
Native Unity tests	13/13 tests passed (5.523 s)
Static analysis (recorded)	0 high, 0 medium, 27 low findings; check passed
Git repository	Milestone history was maintained incrementally; documentation/source changes after the recorded audit remained uncommitted
Wokwi serial output	Not used as primary verification because output was unreliable
OLED / other visual runtime output	Not treated as fully runtime-verified
DHT22 runtime communication	NOT reliably verified in Wokwi configuration

The static-analysis low findings were primarily style-level C-style-cast notices and cross-source unused-function reports that did not correspond to a functional defect in the application. No high- or medium-level findings were recorded in the reviewed implementation.

5.1 Final Unit-Test Result

The final native test run completed successfully with all 13 test cases passing: `test_display_mode`, `test_system_state`, and `test_temperature` all reported PASSED. The complete run finished in approximately 5.5 seconds.

5.2 Wokwi Functional-Test Status

The Wokwi simulation was configured with all required hardware components (DHT22, LDR, PIR, encoder, OLED, buzzer). The simulation started successfully and printed the startup messages "BCA182 FreeRTOS Multisensor" and "System starting..." to the serial monitor.

However, the following runtime behaviors were NOT reliably verified through Wokwi:

- DHT22 runtime communication (logic analyzer showed approximately 5 us pulse instead of expected ms-scale response)
- OLED display output (not reliably observable)
- Serial output beyond startup messages
- Encoder navigation interaction
- Buzzer alarm activation

These limitations are documented honestly rather than claiming verification that could not be reliably observed.

5.3 Problems and Limitations

During development, the main difficulties were unreliable Wokwi serial output and limited visibility of OLED/other visual outputs, which prevented those features from being treated as fully runtime-verified. The native unit-test setup also required a separate host compiler and small HAL/FreeRTOS mocks. The LDR reports a relative 0-100 light level rather than calibrated lux. DHT22 runtime communication could not be reliably verified in the current STM32F103C8 + STM32 HAL + FreeRTOS + Wokwi configuration.

6. Fault Experiments

Three controlled fault experiments were performed to demonstrate the importance of FreeRTOS scheduling and synchronization primitives. Each experiment temporarily introduced a single fault, analyzed the theoretical effect, then restored the original code. Runtime behavior could not be reliably observed in the current Wokwi configuration, so the results are presented as theoretical analysis based on FreeRTOS scheduling principles.

6.1 Experiment 1: Remove Blocking Delay

Item	Details
Target	TaskA (src/main.cpp)
Fault	vTaskDelay(pdMS_TO_TICKS(1000)) temporarily commented out
Expected theoretical effect	TaskA enters a tight loop without yielding, monopolizing CPU time and starving lower-priority TaskB (priority 1). Same-priority tasks experience degraded timing due to timeslice competition. The onboard LED (PC13) would toggle at maximum speed instead of once per second.
Observed effect	Runtime behavior could not be reliably observed in Wokwi. Theoretical analysis confirms CPU monopolization and task starvation.
Restoration	Original delay restored. No permanent changes.

Technical explanation: FreeRTOS is a preemptive, priority-based scheduler. A task blocks when it calls an appropriate blocking API such as vTaskDelay or a queue/semaphore operation. Without a blocking call, TaskA continues consuming CPU time that would otherwise be available to other tasks.

6.2 Experiment 2: Excessively High Task Priority

Item	Details
Target	TaskB (src/main.cpp)
Fault	Priority temporarily changed from 1 to 5 (configMAX_PRIORITIES - 1)
Expected theoretical effect	TaskB can preempt lower-priority tasks, including MotionTask (priority 3), whenever it becomes ready. A diagnostic print task therefore receives an unnecessarily high scheduling priority and can disrupt intended timing.
Observed effect	Runtime behavior could not be reliably observed in Wokwi. Theoretical analysis confirms priority disruption and timing degradation.
Restoration	Original priority (1) restored. No permanent changes.

Technical explanation: In FreeRTOS, a higher-priority ready task preempts a lower-priority task. Raising TaskB to the maximum priority demonstrates how an excessively high priority can disrupt the intended scheduling design.

6.3 Experiment 3: Remove Serial Mutex

Item	Details
Target	serialMutex in Serial_Print() (src/main.cpp)
Fault	xSemaphoreTake() and xSemaphoreGive() temporarily bypassed
Expected theoretical effect	Multiple tasks can call HAL_UART_Transmit() concurrently, causing interleaved or corrupted serial output. Message order becomes nondeterministic depending on scheduling decisions.
Observed effect	Runtime behavior could not be reliably observed in Wokwi. Theoretical analysis confirms output corruption and nondeterminism.
Restoration	Original mutex protection restored. No permanent changes.

Technical explanation: A mutex provides mutual exclusion—only one task can hold the mutex at a time. Without it, multiple tasks can write to the UART hardware simultaneously, allowing byte-level interleaving in transmitted data.

7. Static Code Analysis

PlatformIO static analysis was used to inspect the project for common coding and structural issues. The reviewed implementation produced no high- or medium-severity findings. The remaining low-level findings were mainly style-related and tool-generated cross-source reports. These were reviewed rather than ignored blindly.

8. Engineering Discussion

The project demonstrates the separation of responsibilities expected in a real-time embedded system. Sensor acquisition is isolated from presentation and alarm logic, while FreeRTOS provides scheduling and inter-task communication. `vTaskDelayUntil()` is appropriate for the periodic sensor workload because it maintains a regular sampling schedule without busy waiting.

The queue prevents sensor producers and consumers from depending on unprotected shared sensor variables. The task notification provides a lightweight event mechanism for motion activity. Mutex protection of display selection prevents concurrent access to the navigation state. Critical sections protect short shared-state snapshots where interrupt-disabled atomic access is required.

The principal verification limitation is the current Wokwi environment: serial output was unreliable and OLED/other visual outputs could not be treated as consistently observable. Therefore, the report does not claim full runtime verification for outputs that could not be reliably observed. The DHT22 runtime communication was not reliably verified. The LDR value is also a relative simulated light level rather than calibrated lux.

Git history was maintained incrementally. The major milestone commits include Milestone 1 sensor subsystem, Milestone 2 PIR/activity state, Milestones 3-5 input/display/alarm, Milestone 6 native unit tests, and the final production-build cleanup commit.

9. Requirements Traceability Matrix

The following matrix maps each functional requirement to its implementation, verification method, and current status. Status distinguishes between code-level implementation and actual runtime verification.

Requirement	Implementation	Verification / Test	Status
FR-01 Temperature	DHT22_Read() in sensors.cpp	Native: not applicable; Wokwi: NOT verified	Implemented; NOT runtime verified
FR-02 Humidity	DHT22_Read() in sensors.cpp	Native: not applicable; Wokwi: NOT verified	Implemented; NOT runtime verified
FR-03 Light level	LDR_Read() in sensors.cpp; ADC1 CH0, 0-100 scale	Native: not applicable; Code review verified	Implemented; Code verified
FR-04 Motion	PIR_IsMotionDetected() in motion.cpp	Native: not applicable; Wokwi: NOT verified	Implemented; NOT runtime verified
FR-05 OLED display	DisplayTask in display.cpp; SSD1306 I2C driver	Native: not applicable; Wokwi: NOT verified	Implemented; NOT runtime verified
FR-06 Encoder nav	InputTask in input.cpp; NextDisplayMode / PreviousDisplayMode	Native: 4 tests PASS; display mode navigation	Implemented; Unit tested
FR-07 Temp alarm	EvaluateTemperature() in alarm.cpp	Native: 5 tests PASS; temperature evaluation	Implemented; Unit tested
FR-08 ACTIVE state	EvaluateSystemState() in system_state.cpp	Native: 4 tests PASS; system state evaluation	Implemented; Unit tested
FR-09 Inactivity	15 s timeout via ulTaskNotifyTake()	Native: tested via EvaluateSystemState()	Implemented; Unit tested
FR-10 PIR reactivate	xTaskNotifyGive() from MotionTask	Native: tested via EvaluateSystemState()	Implemented; Unit tested

Status Legend:

- Implemented: Code exists in the production firmware.
- Unit tested: Verified through the native Unity test suite (13/13 PASS).
- Code verified: Correctness confirmed through code review and static analysis.
- NOT runtime verified: Could not be reliably observed in Wokwi simulation.
- DHT22 runtime communication was NOT reliably verified in the current configuration.

10. Conclusion

The laboratory activity resulted in a FreeRTOS-based multisensor room monitoring system for the STM32F103C8T6 Blue Pill. The implementation integrates environmental sensing, motion detection, activity-state management, rotary navigation, OLED presentation, and temperature alarm logic using STM32 HAL and native FreeRTOS APIs.

The project demonstrates periodic scheduling, queues, task notifications, mutex protection, critical sections, state-machine logic, unit testing, static analysis, and incremental Git-based development. Three controlled fault experiments were performed to demonstrate the importance of blocking delays, appropriate task priorities, and mutex protection. All faults were restored after observation.

Runtime limitations in Wokwi are explicitly documented so that the report distinguishes implemented functionality from functionality that could not be reliably observed in simulation. The native unit tests (13/13 PASS) verify the core decision logic for temperature evaluation, display-mode navigation, and system-state transitions.

The requirements traceability matrix confirms that all functional requirements (FR-01 through FR-10) are implemented in the production firmware, with FR-06, FR-07, FR-08, FR-09, and FR-10 additionally verified through native unit tests. Runtime verification of DHT22 communication, OLED output, and other visual outputs remains limited by the current Wokwi simulation environment.

Appendix A - Project Structure

```
+-- include/
| +-- alarm.h
| +-- display.h
| +-- FreeRTOSConfig.h
| +-- input.h
| +-- motion.h
| +-- sensors.h
| +-- system_state.h
+-- src/
| +-- alarm.cpp # AlarmTask + EvaluateTemperature()
| +-- display.cpp # DisplayTask + SSD1306 driver
| +-- hal_mock.cpp # Test-only stubs
| +-- input.cpp # InputTask + NextDisplayMode/PreviousDisplayMode
| +-- main.cpp # Entry point, hardware init, task creation
| +-- motion.cpp # MotionTask + PIR driver
| +-- sensors.cpp # DHT22 bit-bang + LDR ADC read
| +-- system_state.cpp # StateTask + EvaluateSystemState()
+-- test/ # Native Unity tests: 5 + 4 + 4
+-- lib/FreeRTOS/ # FreeRTOS kernel sources (ARM_CM3 port)
+-- docs/
| +-- laboratory-report.pdf
| +-- images/
| +-- system-architecture.png
| +-- task-communication.png
| +-- state-machine.png
+-- diagram.json # Wokwi circuit diagram
+-- wokwi.toml # Wokwi firmware paths
+-- platformio.ini # Build configuration
+-- README.md
```

Appendix B - Build Results

Recorded build result: SUCCESS — RAM approximately 78.1% (15,988 / 20,480 bytes), Flash approximately 28.1% (18,428 / 65,536 bytes). Static analysis: 0 HIGH, 0 MEDIUM, 27 LOW findings; the low-severity findings were reviewed as style/tool reports.